16 x 8 Synchronous Dual-Port RAM

A PROJECT REPORT

Submitted to

NIT SRINAGAR

by

SAJEEL ABID PARRAY

Bearing college enrolment

2026MECEME012

Pursuing

M. TECH

in

MICROELECTRONICS



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

NIT SRINAGAR, HAZRATBAL SRINAGAR

NOVEMBER, 2026

NATIONAL INSTITUTE OF TECHNOLOGY

Hazratbal Srinagar - 190006

CONTENTS

Title page no.

1. INTRODUCTION…… 3

2. BLOCK DIAGRAM…… 4

2.1 Signals and Specification…… 4

2.2 Functionality……... 4

3. VERILOG CODES…….. 6

3.1 RTL code and explanation…….. 6

3.2 Testbench code and explanation……. 8

3.3 GitHub link……. 11

4. RESULTS…….. 12

4.1 Waveforms…….. 12

4.2 Console output……..... 12

5. APPLICATIONS…….. 14

5.1 Current developments……. 14

6. CONCLUSION…… 16

7. REFERENCES…… 17

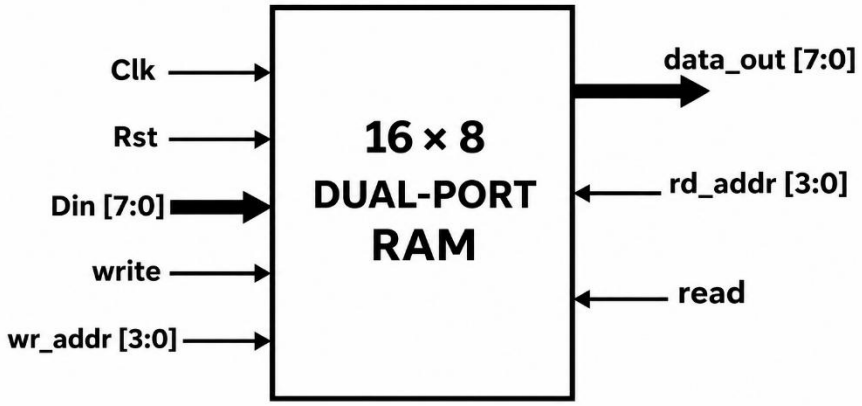
1. INTRODUCTION

Random Access Memory (RAM) is a volatile semiconductor memory used for temporarily storing digital data in electronic systems. It allows data to be accessed directly from any memory location without accessing the preceding locations, making read and write operations faster and more efficient. RAM is widely used in processors, digital systems, and embedded applications for temporary data storage during system operation.

In this project, a **16 × 8 Synchronous Dual-Port RAM** is designed using Verilog HDL. The memory consists of 16 addressable locations, each capable of storing 8 bits of data, giving a total storage capacity of 128 bits. It uses separate 4-bit read and write address inputs, allowing independent selection of any of the 16 memory locations for reading and writing. An 8-bit data input (**Din**) is used to write data into the memory, while an 8-bit data output (**data_out**) provides the stored data during read operations.

The design operates synchronously with a common clock signal (**Clk**), with dedicated read enable and write enable control signals to manage memory operations. A reset (**Rst**) signal is also provided to initialize the design. This architecture allows read and write operations to be controlled independently through separate address inputs, making it suitable for applications requiring efficient memory access and data handling.

2. BLOCK DIAGRAM



2.1 SIGNALS AND SPECIFICATIONS

Clk: The clock signal synchronizes the read and write operations of the RAM. Operations are performed with respect to the active edge of the clock.

Rst: An active high reset signal used to initialize or simply reset the RAM design.

Din [7:0]: Carries the 8-bit data to be written into the selected memory location.

write: Enables the write operation when HIGH (logic 1), allowing input data to be stored at the location specified by the write address.

wr_addr [3:0]: Specifies the address of the memory location where the input data is to be written. It can select any of the 16 memory locations.

data_out [7:0]: Provides the 8-bit data retrieved from the selected memory location during a read operation.

rd_addr [3:0]: Specifies the address of the memory location from which the stored data is to be read.

read: Enables the read operation when HIGH (logic 1), allowing the data stored at the selected read address to be transferred to the output.

2.2 FUNCTIONALITY

The RAM operates synchronously with the **Clk** (Clock) signal, which controls the timing of memory read and write operations. An active-high **Rst** (Reset) signal is provided to initialize the design when asserted.

- **Write Operation:** When the write enable signal (**write**) is HIGH, the RAM is enabled for writing. The 4-bit **wr_addr** (Write Address) signal is used to select a memory location out of 16 locations where the data is to be stored. The 8-bit data supplied through **Din** (Data Input) is written into the selected memory location at the active clock edge. If the write enable is LOW, no write operation takes place, and the previously stored data remains unchanged.

- **Read Operation:** When the read enable signal (**read**) is HIGH, the RAM is enabled for reading. The 4-bit **rd_addr** (Read Address) signal is used to select a memory location out of 16 locations from where data is to

be retrieved. At the active clock edge, the 8-bit data stored at the selected address is transferred to **data_out** (Data Output). When the read enable is LOW, no new read operation takes place.

- **Independent Read and Write Mechanism:** The design uses separate read and write address signals (**rd_addr and wr_addr**) along with independent **read** and **write** enable signals. This allows a read operation and a write operation to take place simultaneously at different memory locations within the same clock cycle. Furthermore, the RTL implements a write-first mechanism for simultaneous read and write operations targeting the same address. In this condition, the newly supplied input data (**Din**) is directly forwarded to **data_out**.

3. VERILOG CODES

3.1 RTL Code and Explanation

```
14  module Sync_RAM_16x8_dualport (
15
16  input Clk, read, write, Rst,          // Control signals for clock, read enable, write enable and reset respectively
17  input [7:0] Din,                      // 8-bit data input for writing into the RAM
18  input [3:0] rd_addr, wr_addr,         // 4-bit read and write addresses for accessing the RAM locations
19  output reg [7:0] data_out          );    // 8-bit data output for reading from the RAM
20
21  parameter RAM_WIDTH = 8,             // Defining parameters
22            RAM_DEPTH = 16;
23
24  reg [RAM_WIDTH-1 :0] mem [RAM_DEPTH-1 :0];       // 2D array representing the RAM with 16 locations of 8 bit wide each
25
26  always @(posedge Clk)                   // Always block triggered on the positive edge of the clock signal
27  begin
28     if (Rst)                             // If reset signal is asserted it resets the output data register (data_out) to zero
29     data_out <= 8'd0;
30     else
31     begin
32       if(write)                // If write enable signal is asserted, write the input data into the RAM at the specified memory location...
33       mem[wr_addr] <= Din;     // ...specified by the write address (wr_addr)
34
35  /* If read signal is asserted, it reads the data from the RAM from the specified address location specified by the read address (rd_addr) and assigns the data
36  to the output register (data_out) normally .But if both read and write signals are asserted simultaneously and the read and write addresses are the same,
37  the output data will be passed the input data (Din) */
38
39       if(read)
40       data_out <= (write && (wr_addr == rd_addr)) ? Din : mem[rd_addr];
41     end
42  end
43
44  endmodule
```

Explanations:

```
module Sync_RAM_16x8_dualport (

input Clk, read, write, Rst,
input [7:0] Din,
input [3:0] rd_addr, wr_addr,
output reg [7:0] data_out

parameter RAM_WIDTH = 8,
          RAM_DEPTH = 16;
```

- **module** defines the RAM module named **Sync_RAM_16x8_dualport**.
- **input Clk, read, write, Rst** declares the 1-bit clock, read enable, write enable and reset signal.
- **input [7:0] Din** declares an 8-bit data input bus.
- **input [3:0] rd_addr, wr_addr** declares separate 4-bit read and write address buses, selecting one of the 16 memory locations.
- **output reg [7:0] data_out** declares an 8-bit registered output. **reg** is used because it is assigned inside a procedural block.
- **parameter** defines constant values for the memory dimensions.
- **RAM_WIDTH = 8** specifies 8 bits per memory location.
- **RAM_DEPTH = 16** specifies 16 memory locations.

```
reg [RAM_WIDTH-1 :0] mem [RAM_DEPTH-1 :0];
```

- Declares the memory array named **mem** using the **reg** datatype.
- **[RAM_WIDTH-1:0]** defines an 8-bit width for each location.
- **[RAM_DEPTH-1:0]** defines 16 addressable locations, indexed from 15 to 0.
- Therefore, the total memory capacity is 16 × 8 = 128 bits.

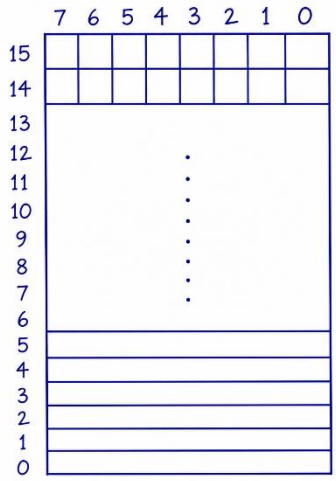


Fig. 3.1 2-D Memory array representation.

```
always @(posedge Clk)                    // Always block triggered on t
begin
   if (Rst)                              // If reset signal is asserted
   data_out <= 8'd0;
   else
   begin
     if(write)                 // If write enable signal is asserted, w
     mem[wr_addr] <= Din;      // ...specified by the write address (wr

/* If read signal is asserted, it reads the data from the RAM from the
to the output register (data_out) normally .But if both read and write
the output data will be passed the input data (Din) */

     if(read)
     data_out <= (write && (wr_addr == rd_addr)) ? Din : mem[rd_addr];
   end
end
```

- **always** defines a procedural block.
- **@(posedge Clk)** triggers the block at every rising edge of the clock. This makes the RAM synchronous, with its operations controlled by the clock.
- **" <= "** is the non-blocking assignment operator, commonly used for sequential logic.
- Reset has priority over read and write operations.
- If **Rst** signal is HIGH or logic 1, **data_out** is assigned 8-bit zeros.
- Otherwise if **Rst** is LOW and if **write** signal is HIGH

mem[wr_addr] <= Din stores the 8-bit input data at the memory location specified by **wr_addr**.

- Or in the special case where both **read** and **write** signals are HIGH and **wr_addr** is equal to **rd_addr**, the condition **((wr_addr == rd_addr) && write)** evaluates to TRUE (logic 1). Consequently, the supplied input data (**Din**) is directly forwarded to **data_out** instead of reading the previously stored data from memory. This implements a write-first mechanism. Otherwise, when **read** is HIGH and the above condition is FALSE, the normal read operation is performed, and the stored data from **mem[rd_addr]** is transferred to **data_out**.
- The ternary operator **(? :)** selects the data to be transferred to **data_out**. If **((wr_addr == rd_addr) && write)** statement is TRUE it assigns the first assignment left to **:** elsewise the second one on the right of **:**.
- If **read** is LOW, the output retains its previous value.
- Separate read and write enable signals and address buses allow both operations to occur on the same rising clock edge.

3.2 Testbench and Explanation

```
13  `timescale 1ns/ 1ps                    // defining time unit and time precision
14
15  module Sync_RAM_16x8_dualport_tb ();
16
17  reg Clk, Rst, read, write;          // Making Inputs as reg type to drive the signals in the testbench...
18  reg [7:0] Din;                      // ... and outputs as wire type to monitor the signals in the testbench
19  wire [7:0] data_out;
20  reg [3:0] rd_addr, wr_addr;
21  integer i;                          // declaring integer variable for loop iteration
22
23  Sync_RAM_16x8_dualport DUT (.Clk(Clk), .Rst(Rst), .read(read), .write(write), .Din(Din),  // Module instantiation and mapping the signals in the...
24                              .data_out(data_out), .rd_addr(rd_addr), .wr_addr(wr_addr));   //...testbench to the module ports
25
26  always              // Always block to generate clock signal with a period of 10ns
27    begin
28      Clk = 0; #5;
29      Clk = 1; #5;
30    end
31
32  task reset();       // Task to perform reset operation on the RAM and at the same time initialization of the input signals.
33  begin
34    @(negedge Clk)
35    Rst= 1;
36    Din =0;
37    write =0;
38    read=0;
39    wr_addr =0;
40    rd_addr =0;
41    @(posedge Clk)
42    Rst=0;
43  end
44  endtask
46  task write_t (input [3:0] w_addr, input [7:0] data);       // Task to write data into the RAM
47  begin
48    @(negedge Clk)
49    write = 1;
50    read = 0;
51    wr_addr = w_addr;
52    Din = data;
53  end
54  endtask
55
56  task read_t (input [3:0] r_addr);         // Task to read data from the RAM
57  @(negedge Clk)
58  begin
59    write = 0;
60    read = 1;
61    rd_addr = r_addr;
62  end
63  endtask
65  initial
66  begin
67    reset;
68
69    // Basic write and read
70    write_t (4'd3, 8'd20);          // Write 20 to address 3(0011)
71    read_t  (4'd3);                 // Read from address 3(0011)
72
73    // Overwrite same address
74    write_t (4'd3, 8'd15);         // Write 15 to address 3(0011)
75
76    // Write to another address
77    write_t (4'd7, 8'd8);          // Write 8 to address 7(0111)
78
79    // read & write on same address
80    @(negedge Clk);
81    write   = 1;
82    read    = 1;
83    wr_addr = 4'd7;
84    rd_addr = 4'd7;
85    Din     = 8'd24;
```

```
86
87    // Full address sweep
88    for (i = 0; i < 16; i = i + 1)
89    begin
90      write_t (i, {$random % 256});        // Loop to write random data to all addresses and then read from them
91      read_t  (i);
92    end
93
94    //read and write on different addresses at the same time (dual-port behavior)
95    @(negedge Clk);
96    write   = 1;
97    read    = 1;
98    wr_addr = 4'd2;
99    rd_addr = 4'd9;
100   Din     = 8'd55;
101   @(negedge Clk);      // read from address 9(1001) )
102   write   = 0;
103   read    = 1;
104   rd_addr = 4'd2;
105
106   #100;
107   $finish;           // Finish the simulation after 100ns
108 end
109
110 // Monitor block to display the values of the signals at each time step
111 initial $monitor ("Time= %0t, Clk= %b, Rst= %b, write= %b, read= %b, Din= %0d, wr_addr= %0d, rd_addr= %0d, data_out= %0d",
112                   $time, Clk, Rst, write, read, Din, wr_addr, rd_addr, data_out);
113
114 endmodule
```

EXPLANATIONS:

```
13  `timescale 1ns/ 1ps
```

- Defines the simulation time unit as 1 ns and time precision as 1 ps.
- All simulation delays, such as **#5**, are interpreted according to this timescale.

```
15  module Sync_RAM_16x8_dualport_tb ();
16
17  reg Clk, Rst, read, write;
18  reg [7:0] Din;
19  wire [7:0] data_out;
20  reg [3:0] rd_addr, wr_addr;
21  integer i;
```

- module **Sync_RAM_16x8_dualport_tb ();** declares the testbench module named Sync_RAM_16x8_dualport_tb.
- No input or output ports are required because the testbench internally generates and monitors all signals.

- **reg** declares signals that are assigned values procedurally within the testbench.
- **Clk**, **Rst**, **read**, and **write** are declared as 1-bit control signals.
- **Din** is an 8-bit register used to supply input data to the RAM.
- **rd_addr** and **wr_addr** are 4-bit registers used to select memory locations.
- **data_out** is declared as a wire because it receives its value from the RAM module.
- **integer i** is used as a loop variable for iterating through all 16 memory addresses.

```
23  Sync_RAM_16x8_dualport DUT (.Clk(Clk), .Rst(Rst), .read(read), .write(write), .Din(Din),
24                              .data_out(data_out), .rd_addr(rd_addr), .wr_addr(wr_addr));
```

- Instantiates the RAM module inside the testbench with the instance name DUT (Design Under Test).
- Connects the testbench signals to their corresponding ports in the RAM module using named port mapping.
- This establishes communication between the testbench and the actual RAM design.

```
26   always
27     begin
28       Clk = 0; #5;
29       Clk = 1; #5;
30     end
```

- An infinite always block generates a continuous clock signal.
- **Clk = 0** keeps the clock LOW for **5 ns**, followed by **Clk = 1** for another **5 ns**.
- This generates a clock with a 10 ns time period.

```
32  task reset();
33  begin
34    @(negedge Clk)
35    Rst= 1;
36    Din =0;
37    write =0;
38    read=0;
39    wr_addr =0;
40    rd_addr =0;
41    @(posedge Clk)
42    Rst=0;
43  end
44  endtask
```

- **task reset();** defines a reusable task named reset to initialize the RAM and
- testbench input signals.
- **@(negedge Clk)** waits for the falling edge of the clock before applying the reset.
- **Rst = 1** activates the active-high reset, while all data, address and enable signals are initialized to zero.

Note: We are providing inputs before the positive edge (negedge) and DUT samples them at positive edge (posedge).

- **@(posedge Clk)** waits for the next rising clock edge, after which **Rst** is de-asserted by assigning zero.

```
46  task write_t (input [3:0] w_addr, input [7:0] data);
47  begin
48    @(negedge Clk)
49    write = 1;
50    read = 0;
51    wr_addr = w_addr;
52    Din = data;
53  end
54  endtask
```

- **task write_t** defines a reusable task named write_t with two inputs **w_addr** and **data** limited to the task, mapped to our **wr_addr** and **Din** respectively by **wr_addr = w_addr;** and **Din = data;**
- **@(negedge Clk)** waits for the falling edge of the clock before applying the inputs.
- Sets **write = 1** to enable writing and **read = 0** to disable reading.

```
56   task read_t (input [3:0] r_addr);
57   @(negedge Clk)
58   begin
59     write = 0;
60     read = 1;
61     rd_addr = r_addr;
62   end
63   endtask
```

- **task read_t** defines a reusable task read_t that accepts a 4-bit read address **r_addr** mapped to **rd_addr** signal.
- Waits for the falling edge of the clock before applying the read inputs.
- Sets **write = 0** to disable writing and **read = 1** to enable reading.

```
65  initial
66  begin
67    reset;
68
69    // Basic write and read
70    write_t (4'd3, 8'd20);
71    read_t  (4'd3);
72
73    // Overwrite same address
74    write_t (4'd3, 8'd15);        /
75
76    // Write to another address
77    write_t (4'd7, 8'd8);         /
78
79    // read & write on same address
80    @(negedge Clk);
81    write   = 1;
82    read    = 1;
83    wr_addr = 4'd7;
84    rd_addr = 4'd7;
85    Din     = 8'd24;
86
87    // Full address sweep
88    for (i = 0; i < 16; i = i + 1)
89    begin
90      write_t (i, {$random % 256});
91      read_t  (i);
92    end
```

- Inside or initial block is our stimulus to the DUT which executes once at the beginning of the simulation.
- **reset** calls the reset task to initialize the RAM and testbench signals before starting the test sequence.
- **write_t (4'd3, 8'd20)** writes decimal value 20 into memory location 3.
- **read_t** subsequently reads from the same address to verify the stored data.
- At negative edge of the **Clk** both read and write operations are simultaneously performed.
- **for** loop is used to iterate through all 16 memory locations, from address 15 to address 0.
- **{$random % 256}** generates a random value within the intended 8-bit range (0–255).
- At each iteration, random data is written into the selected address and subsequently read from the same address.
- This tests the read and write functionality across the entire memory array.

```
95    @(negedge Clk);
96    write   = 1;
97    read    = 1;
98    wr_addr = 4'd2;
99    rd_addr = 4'd9;
100   Din     = 8'd55;
101   @(negedge Clk);
102   write   = 0;
103   read    = 1;
104   rd_addr = 4'd2;
105
106   #100;
107   $finish;
108 end
```

- At negative edge of clock both read and write operations are performed simultaneously.
- Address 2 is assigned for writing and address 9 for reading.
- Decimal value 55 is written into memory location 2 while retrieving the previously stored data from memory location 9 on the same rising clock edge.
- At next negative edge write is disabled while read remains enabled and at address 2 the newly written value (55) is retrieved, thereby verifying the write operation.
- **#100** introduces a simulation delay of 100 ns.
- **$finish** terminates the simulation after the specified delay.

```
111 initial $monitor ("Time= %0t, Clk= %b, Rst= %b, write= %b, read= %b, Din= %0d, wr_addr= %0d, rd_addr= %0d, data_out= %0d",
112                   $time, Clk, Rst, write, read, Din, wr_addr, rd_addr, data_out);
```

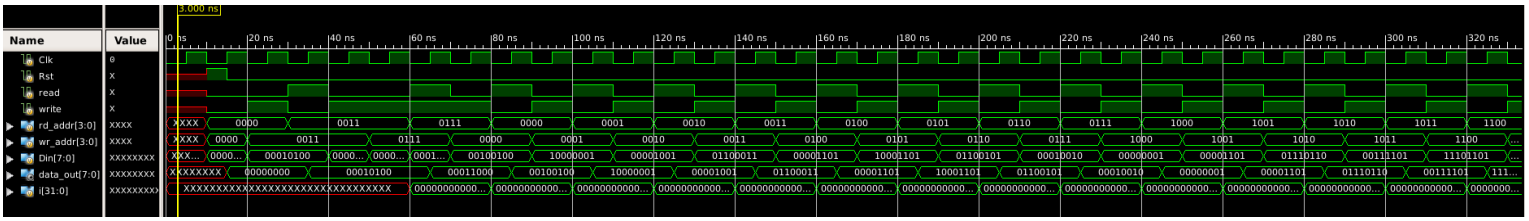
- **$monitor** is a system task which continuously displays the specified signal values whenever any of the monitored signal changes.
- **$time** displays the current simulation time.
- **%b** displays binary values, **%0d** displays decimal values, and **%0t** displays simulation time.
- This allows the read and write operations, address selection, input data and output data to be observed throughout the simulation.

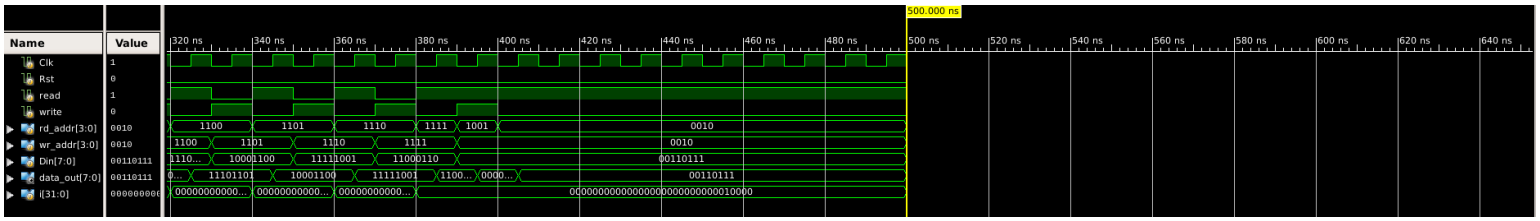
3.3 GitHub Link: github.com/Sajeel-Abid-Parray/M.Tech-/tree/main/ACA

4. RESULTS

The design has been simulated using Xilinx ISE, and the results presented below demonstrate the expected functionality of the design through simulation waveforms and console outputs.

4.1 Waveforms:





4.2 Console Output:

Time= 0, Clk= 0, Rst= x, write= x, read= x, Din= x, wr_addr= x, rd_addr= x, data_out= x

Time= 5000, Clk= 1, Rst= x, write= x, read= x, Din= x, wr_addr= x, rd_addr= x, data_out= x

Time= 15000, Clk= 1, Rst= 0, write= 0, read= 0, Din= 0, wr_addr= 0, rd_addr= 0, data_out= 0

Time= 25000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 20, wr_addr= 3, rd_addr= 0, data_out= 0

Time= 35000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 20, wr_addr= 3, rd_addr= 3, data_out= 20

Time= 45000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 15, wr_addr= 3, rd_addr= 3, data_out= 20

Time= 55000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 8, wr_addr= 7, rd_addr= 3, data_out= 20

Time= 65000, Clk= 1, Rst= 0, write= 1, read= 1, Din= 24, wr_addr= 7, rd_addr= 7, data_out= 24

Time= 75000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 36, wr_addr= 0, rd_addr= 7, data_out= 24

Time= 85000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 36, wr_addr= 0, rd_addr= 0, data_out= 36

Time= 95000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 129, wr_addr= 1, rd_addr= 0, data_out= 36

Time= 105000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 129, wr_addr= 1, rd_addr= 1, data_out= 129

Time= 115000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 9, wr_addr= 2, rd_addr= 1, data_out= 129

Time= 125000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 9, wr_addr= 2, rd_addr= 2, data_out= 9

Time= 135000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 99, wr_addr= 3, rd_addr= 2, data_out= 9

Time= 145000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 99, wr_addr= 3, rd_addr= 3, data_out= 99

Time= 155000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 13, wr_addr= 4, rd_addr= 3, data_out= 99

Time= 165000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 13, wr_addr= 4, rd_addr= 4, data_out= 13

Time= 175000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 141, wr_addr= 5, rd_addr= 4, data_out= 13

Time= 185000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 141, wr_addr= 5, rd_addr= 5, data_out= 141

Time= 195000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 101, wr_addr= 6, rd_addr= 5, data_out= 141

Time= 205000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 101, wr_addr= 6, rd_addr= 6, data_out= 101

Time= 215000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 18, wr_addr= 7, rd_addr= 6, data_out= 101

Time= 225000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 18, wr_addr= 7, rd_addr= 7, data_out= 18

Time= 235000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 1, wr_addr= 8, rd_addr= 7, data_out= 18

Time= 245000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 1, wr_addr= 8, rd_addr= 8, data_out= 1

Time= 255000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 13, wr_addr= 9, rd_addr= 8, data_out= 1

Time= 265000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 13, wr_addr= 9, rd_addr= 9, data_out= 13

Time= 275000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 118, wr_addr= 10, rd_addr= 9, data_out= 13

Time= 285000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 118, wr_addr= 10, rd_addr= 10, data_out= 118

Time= 295000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 61, wr_addr= 11, rd_addr= 10, data_out= 118

Time= 305000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 61, wr_addr= 11, rd_addr= 11, data_out= 61

Time= 315000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 237, wr_addr= 12, rd_addr= 11, data_out= 61

Time= 325000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 237, wr_addr= 12, rd_addr= 12, data_out= 237

Time= 335000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 140, wr_addr= 13, rd_addr= 12, data_out= 237

Time= 345000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 140, wr_addr= 13, rd_addr= 13, data_out= 140

Time= 355000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 249, wr_addr= 14, rd_addr= 13, data_out= 140

Time= 365000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 249, wr_addr= 14, rd_addr= 14, data_out= 249

Time= 375000, Clk= 1, Rst= 0, write= 1, read= 0, Din= 198, wr_addr= 15, rd_addr= 14, data_out= 249

Time= 385000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 198, wr_addr= 15, rd_addr= 15, data_out= 198

Time= 395000, Clk= 1, Rst= 0, write= 1, read= 1, Din= 55, wr_addr= 2, rd_addr= 9, data_out= 13

Time= 405000, Clk= 1, Rst= 0, write= 0, read= 1, Din= 55, wr_addr= 2, rd_addr= 2, data_out= 55

Stopped at time : 500 ns

5. APPLICATIONS

The designed 16 × 8 Synchronous Dual-Port RAM demonstrates a memory architecture in which data can be written and read through separate ports using independent address and enable signals. This type of memory organization is useful in digital systems where simultaneous data storage and retrieval are required. Although the implemented design has a relatively small storage capacity of 128 bits, the same architectural approach can be extended to larger memory systems and practical applications.

Some important real-world applications include:

- **Digital Signal Processing (DSP):** Used for temporary storage of input samples, intermediate results and processed data, allowing data to be written and retrieved efficiently.
- **FIFO Buffers and Data Queues:** Similar dual-port memory architectures are used in buffering systems to manage data transfer between different processing stages.
- **Processor Register Files:** Multiport memory structures based on similar principles are used to provide fast access to stored operands and intermediate results.
- **Communication Systems:** Used in data buffering and packet-processing units, where incoming and outgoing data may need to be accessed simultaneously.
- **FPGA-Based Digital Systems:** Dual-port RAM structures are commonly used for temporary storage, data exchange between functional blocks and intermediate data buffering.
- **Embedded Systems:** Used in controllers and digital devices for temporary data storage and efficient handling of incoming and processed information.

5.1 Current Developments

Modern digital systems require increasingly faster memory access, higher storage density and lower power consumption. Consequently, similar dual-port memory architectures are being developed and integrated into advanced FPGA and ASIC technologies.

- FPGA Embedded Memory: Modern FPGA devices, including AMD Xilinx architectures, provide dedicated Block RAM (BRAM) resources that can be configured in Simple Dual-Port and True Dual-Port modes. These allow simultaneous read and write operations without requiring the entire memory to be constructed from individual logic elements.

- Advanced Memory Configurations: Modern FPGA memory architectures support configurable data widths, memory depths and different clocking arrangements. Some implementations also support separate read and write clocks, allowing data exchange between different clock domains.

- Read-During-Write Modes: Modern memory blocks provide different operating modes, such as WRITE_FIRST, READ_FIRST and NO_CHANGE, to determine the output behaviour when a read and write operation access the same memory location simultaneously. These modes provide flexibility in designing memory systems for different applications.

- High-Performance Memory Systems: Similar memory architectures are increasingly used in processor register files, cache structures, DSP units and high-speed communication systems. More advanced designs employ multiple memory banks and additional ports to increase memory bandwidth and support parallel data processing.

Relation to the Proposed Design: The implemented 16 × 8 RAM follows the same fundamental approach used in modern dual-port memory architectures. It uses separate read and write address buses, independent enable signals and clock-synchronized operations. Its write-first mechanism also represents one of the read-during-write behaviours supported by modern FPGA memory resources. Thus, the project provides a basic RTL-level implementation of a memory architecture that is widely used in contemporary digital hardware design.

6. CONCLUSION

In this project, a **16 × 8 Synchronous Dual-Port RAM** was successfully designed and simulated using Verilog HDL in Xilinx ISE. The design consists of 16 memory locations, each capable of storing 8 bits of data, providing a total storage capacity of 128 bits. Separate 4-bit read and write address buses, along with independent read and write enable signals, were implemented to control memory operations.

The design successfully demonstrates clock-synchronized read and write operations, active-high reset functionality, and simultaneous access to different memory locations. Additionally, a write-first mechanism was implemented to handle simultaneous read and write operations targeting the same address, ensuring that the newly supplied input data is directly forwarded to the output.

A dedicated testbench was developed to verify the functionality of the design under different operating conditions, including basic read and write operations, overwriting existing data, accessing all memory locations, and simultaneous read and write operations at both identical and different addresses. The simulation waveforms and console outputs confirmed the expected behaviour of the implemented design.

Overall, this project provided practical understanding of RAM architecture, synchronous memory operations, Verilog HDL coding, RTL design and functional verification. It also established a foundation for developing more advanced memory architectures, such as larger dual-port RAMs and True Dual-Port RAMs, for FPGA and ASIC-based digital systems.

7. REFERENCES

[1] A. Pandey, B. Suhas, B. P. and S. Madhavan, "Design and Implementation of Synchronous Dual-Port Memory," *2024 Third International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*, April 2024.

[2] EcrioniX, "Simple Dual Port RAM — Verilog Guide," *Digital Electronics*.

[3] AMD Xilinx, UltraScale Architecture Memory Resources User Guide (UG573), *Chapter 1: Block RAM Resources*.

[4] AMD Xilinx, Vivado Design Suite User Guide: Synthesis (UG901), *Simple Dual-Port Block RAM Examples*.

[5] AMD Xilinx, Vivado Design Suite Reference Guide: Model-Based DSP Design Using System Generator (UG958), *Dual Port RAM*.